

SERVICIO NACIONAL DE APRENDIZAJE – SENA
Centro de Gestión de Mercados, Logística y TIC

APP TRUEQUE MATCH

Evidencia GA7-220501096-AA5-EV01. Diseño y desarrollo de servicios web - caso.

Estudiante: Gerson Jonnathan Lopez Oviedo

Ficha: 3186647

Instructor: Arnaldo Alfonso Montiel Brun

Programa: Tecnólogo en Análisis y Desarrollo de Software

Bogotá. Colombia 2026

Parte 1: Desarrollo del Formulario CRUD

1. Introducción

En esta primera parte del proyecto, desarrolle un sistema de gestión de usuarios utilizando PHP, MySQL y XAMPP. El sistema permite realizar las cuatro operaciones fundamentales de cualquier base de datos: crear, leer, actualizar y eliminar registros. A este conjunto de operaciones se le conoce en el mundo del desarrollo de software como CRUD (por sus siglas en inglés: Create, Read, Update, Delete).

Para construir el sistema, cree una carpeta llamada formulario dentro de la ruta C:\xampp\htdocs\, que es la carpeta raíz que XAMPP utiliza para servir los proyectos web a través del navegador.

2. Base de Datos

Cree la base de datos formulario_db directamente desde phpMyAdmin (accesible en localhost/phpmyadmin). Dentro de esta base de datos, cree la tabla usuarios con las siguientes columnas:

Columna	Tipo de Dato	Característica	Descripción
id	INT	PK, Auto Increment	Identificador único automático
nombre	VARCHAR(100)	Requerido	Nombre del usuario registrado
email	VARCHAR(100)	Requerido	Correo electrónico del usuario

Nota: el servidor MySQL de XAMPP en mi computador corre en el puerto **3307** (no el puerto 3306 estándar), debido a que otro programa ya ocupaba ese puerto. Esto se configuro en el archivo conexion.php.

3. Archivos Desarrollados

El módulo CRUD está compuesto por los siguientes archivos, cada uno con una función específica:

Archivo	Función
conexion.php	Establece la conexión con la base de datos MySQL. Contiene el servidor, usuario, contraseña, nombre de la BD y el puerto 3307.
index.php	Página principal. Muestra el formulario para registrar usuarios y una tabla con todos los registros existentes.

guardar.php	Recibe los datos del formulario por POST y los inserta en la base de datos con el comando INSERT INTO.
editar.php	Recibe el ID del usuario por GET, lo busca en la BD con SELECT y muestra sus datos precargados en un formulario editable.
actualizar.php	Recibe los datos editados por POST y actualiza el registro en la BD con el comando UPDATE ... WHERE id.
eliminar.php	Recibe el ID del usuario por GET y lo elimina de la BD con el comando DELETE FROM ... WHERE id.
style.css	Hoja de estilos que da el aspecto visual al sistema: colores, fuentes, tabla, botones y centrado de contenido.

4. Funcionamiento del CRUD

4.1 Crear (INSERT)

Desde index.php, lleno el formulario con el nombre y correo del usuario y presiono el botón Registrar. Los datos viajan de forma oculta (method POST) hacia guardar.php, que ejecuta el comando INSERT INTO usuarios (nombre, email) VALUES (...) para guardarlos en la base de datos.

4.2 Leer (SELECT)

En la misma página index.php, después del formulario se muestra una tabla con todos los registros existentes. Esto se logra con el comando SELECT * FROM usuarios, cuyos resultados recorro con un bucle while que imprime una fila por cada usuario.

4.3 Actualizar (UPDATE)

Al presionar el botón Editar de cualquier fila, el sistema lleva a editar.php con el ID del usuario en la URL. Este archivo busca ese registro con SELECT * FROM usuarios WHERE id = X y carga los datos actuales en los campos del formulario. Al guardar, actualizar.php ejecuta UPDATE usuarios SET nombre = ..., email = ... WHERE id = X.

4.4 Eliminar (DELETE)

Al presionar el botón Eliminar, el sistema envía el ID del usuario a eliminar.php, que ejecuta DELETE FROM usuarios WHERE id = X y redirige automáticamente a index.php para mostrar la tabla actualizada.

Parte 2: Sistema de Autenticación (Login)

5. Introducción

En esta segunda parte, implemente un sistema de autenticación para proteger el acceso al módulo CRUD. La idea es que solo los administradores registrados puedan ingresar al sistema. Para esto, utilice sesiones PHP y encriptación de contraseñas con la función password_hash(), que convierte la contraseña en un código irreversible antes de guardarla en la base de datos.

Al conectar ambas partes, el sistema quedo completo: primero hay que iniciar sesión y, solo después de autenticarse correctamente, se puede acceder al formulario CRUD de gestión de usuarios.

6. Tabla de Administradores

Dentro de la misma base de datos formulario_db, cree una segunda tabla llamada cuentas, exclusivamente para guardar las credenciales de los administradores:

Columna	Tipo de Dato	Característica	Descripción
id	INT	PK, Auto Increment	Identificador único automático
usuario	VARCHAR(100)	Único	Nombre de usuario del administrador
contrasena	VARCHAR(255)	Requerido	Hash de la contraseña (bcrypt)

La columna contraseña se definió con VARCHAR(255) (y no VARCHAR(100)) porque el hash generado por password_hash() puede tener hasta 60 caracteres o más, dependiendo del algoritmo utilizado. Usar 255 garantiza compatibilidad con versiones futuras de PHP.

7. Archivos del Sistema de Autenticación

Archivo	Función
registro.php	Muestra el formulario para crear una cuenta nueva de administrador (usuario + contraseña + confirmación).
guardar_cuenta.php	Verifica que las dos contraseñas coincidan, encripta la contraseña con password_hash() y guarda el registro en la tabla cuentas.
login.php	Muestra el formulario de inicio de sesión. Si ya hay una sesión activa, redirige directamente a index.php.
verificar_login.php	Busca el usuario en la BD y verifica la contraseña con password_verify(). Si es correcto, crea la sesión y redirige a index.php.
cerrar_sesion.php	Destruye la sesión activa con session_destroy() y redirige al login.

8. Seguridad Implementada

Para garantizar la seguridad del sistema, implemente las siguientes medidas:

- Encriptación de contraseñas: uso password_hash() con PASSWORD_DEFAULT (algoritmo bcrypt), que genera un hash irreversible. Nadie puede saber la contraseña original mirando la base de datos.
- Verificación segura: uso password_verify() para comparar la contraseña ingresada con el hash guardado, sin necesidad de descriptar nada.

- Protección de páginas: al inicio de index.php agrego session_start() e isset(\$_SESSION['usuario']). Si no hay sesion activa, el usuario es redirigido automáticamente al login.
- Cierre correcto de sesion: cerrar_sesion.php destruye completamente la sesion con session_destroy(), impidiendo que alguien acceda con el botón atrás del navegador.
- Mensajes de error genéricos: cuando el usuario o la contraseña son incorrectos, muestro el mensaje 'Usuario o contraseña incorrectos' sin especificar cuál de los dos fallo, para no dar pistas a posibles intrusos.
- exit() obligatorio: después de cada redirección con header('Location: ...'), agrego exit() para detener la ejecución del código y evitar comportamientos inesperados.

9. Flujo Completo del Sistema

El siguiente diagrama describe el recorrido completo que hace un administrador al usar el sistema:

Paso	Acción del Usuario	Lo que hace el Sistema
1	Intenta entrar a index.php sin sesion	El guardia de sesion lo redirige automáticamente a login.php
2	Ingresa usuario y contraseña incorrectos	Muestra mensaje de error en rojo sin revelar cual campo fallo
3	Ingresa usuario 'admin' y contraseña 'admin123'	password_verify() confirma, crea sesion y redirige a index.php
4	Registra un nuevo usuario en el formulario	INSERT INTO usuarios guarda el registro y actualiza la tabla
5	Edita un usuario existente	SELECT carga los datos, UPDATE guarda los cambios

6	Elimina un usuario	DELETE FROM usuarios WHERE id elimina solo ese registro
7	Cierra sesion	session_destroy() elimina la sesion y regresa al login

Parte 3: Credenciales, Verificación y Repositorio

10. Credenciales de Prueba del Sistema

Para verificar el funcionamiento del sistema, utilice las siguientes credenciales de acceso al panel de administración:

Campo	Valor
Usuario	admin
Contraseña	admin123
Base de datos	formulario_db
Puerto MySQL	3307 (configurado en conexion.php)
URL de acceso	localhost/formulario/login.php

11. Instrucciones para Verificar el Sistema

Se recomienda usar Google Chrome o Microsoft Edge para la verificación. Los pasos son los siguientes:

Requisitos previos

- Tener XAMPP instalado con Apache y MySQL corriendo (MySQL configurado en puerto 3307).

- Descargar el proyecto del repositorio de GitHub y colocarlo en `C:\xampp\htdocs\formulario\`
- Crear la base de datos `formulario_db` en phpMyAdmin con las tablas usuarios y cuentas según la estructura descrita en este documento.
- Registrar una cuenta de administrador en `localhost/formulario/registro.php` con usuario `admin` y contraseña `admin123`.

Flujo de prueba recomendado

- Paso 1: Abrir el navegador e ir a `localhost/formulario/index.php`. El sistema debe redirigir automáticamente a `login.php`.
- Paso 2: Intentar iniciar sesión con datos incorrectos. Debe aparecer el mensaje de error en rojo.
- Paso 3: Iniciar sesión con usuario `'admin'` y contraseña `'admin123'`. Debe aparecer `'Bienvenido, admin'`.
- Paso 4: Registrar un nuevo usuario con nombre y correo. Verificar que aparece en la tabla.
- Paso 5: Editar ese usuario y verificar que los cambios se guardan correctamente.
- Paso 6: Eliminar el usuario y verificar que desaparece de la tabla.
- Paso 7: Hacer clic en `'Cerrar sesión'` y verificar que regresa al login.
- Paso 8: Intentar acceder a `localhost/formulario/index.php` sin sesión. Debe redirigir al login.

12. Repositorio en GitHub

El código fuente completo del proyecto, con comentarios explicativos en cada línea de código (etiquetas HTML, atributos, funciones PHP y comandos SQL), está disponible en el siguiente repositorio de GitHub:

Repositorio del Proyecto: <https://github.com/lopezoviedogersonjonnathan-rgb/formulario-CRUD-de-usuarios-con-PHP-MySQL-y-XAMPP>

Para descargar el proyecto directamente con Git Bash, ejecute el siguiente comando:

```
git clone https://github.com/lopezoviedogersonjonnathan-rgb/formulario-CRUD-de-usuarios-con-PHP-MySQL-y-XAMPP.git
```

El repositorio contiene todos los archivos del proyecto organizados en la carpeta formulario/, listos para ser copiados a la carpeta htdocs de XAMPP y ejecutados localmente.

13. Conclusiones

En este proyecto logre construir desde cero un sistema web completo que integra dos componentes fundamentales en el desarrollo de software: la gestión de datos (CRUD) y la autenticación segura de usuarios (login con sesiones y contraseñas encriptadas).

Aprendí a utilizar PHP para conectarme a una base de datos MySQL, ejecutar comandos SQL (INSERT, SELECT, UPDATE, DELETE), manejar sesiones, encriptar contraseñas con `password_hash()` y verificarlas con `password_verify()`. También aprendí a organizar los archivos del proyecto de forma lógica, a usar Git y GitHub para el control de versiones, y a identificar y corregir errores en el código.

Este proyecto me permitió entender cómo funciona un sistema web real por dentro, desde el formulario que ve el usuario hasta la base de datos que guarda la información de forma segura.